Исследование методов проверки микросервисных архитектур, проектируемых большими языковыми моделями

Дичковский Владимир Андреевич

Аннотация

Автоматическое проектирование микросервисных архитектур с помощью больших языковых моделей позволяет получать варианты топологии из текстовых требований, однако синтаксически корректное представление не доказывает выполнения требований к производительности. В работе формализуется задача проверки таких архитектур и предлагается методика сравнительного исследования трёх уровней валидации: структурных ограничений графа, аналитических моделей теории массового обслуживания и дискретно-событийного моделирования. Рассматриваются условия применимости моделей очередей, конечные буферы, случайная нагрузка и статистическая неопределённость перцентилей задержки. Предложен план экспериментов для сопоставления одношаговой генерации и итеративного исправления по результатам проверок, в том числе для локальных языковых моделей. Определены эталоны, измеряемые показатели и критерии достоверности. Количественная проверка гипотез запланирована, поэтому результаты и утверждения о превосходстве методов пока не приводятся.

Ключевые слова: микросервисная архитектура, большие языковые модели, валидация архитектуры, теория массового обслуживания, дискретно-событийное моделирование, нефункциональные требования, экспериментальное исследование.

1 Введение

Микросервисная архитектура состоит из взаимодействующих компонентов с ограниченными вычислительными ресурсами. Наличие корректной схемы связей не означает, что система будет обслуживать запросы с требуемой задержкой и без перегрузки. Структурную корректность и поведение под нагрузкой необходимо проверять отдельно [1, 2].

Большие языковые модели (Large Language Models, LLM) применяются для преобразования текстовых требований в архитектурные представления [3,4]. Однако их собственная оценка пропускной способности предложенной топологии не заменяет расчёт. Нефункциональные требования (Non-Functional Requirements, NFR) включают ограничения на задержку, пропускную способность, стоимость и доступность.

Здесь сравниваются три уровня проверки: ограничения графа, аналитические модели теории массового обслуживания (TMO) и дискретно-событийное моделирование (Discrete-Event Simulation, DES) запросов и ресурсов с помощью SimPy [5]. Заключение о выполнении ограничения может быть получено только в границах явно заданной модели и статистической неопределённости.

1.1 Почему автоматизация проектирования архитектуры актуальна

Разработка программного продукта начинается не с написания кода, а с выбора структуры его компонентов и их взаимодействий. Для микросервисной системы это особенно важно: возможны десятки комбинаций границ сервисов, маршрутов запросов, количества реплик, политик кэширования и способов обмена сообщениями. Разные варианты могут обеспечивать одну и ту же функциональность, но существенно различаться по задержке, устойчивости к пиковому трафику, стоимости и сложности эксплуатации. Ручное сопоставление всех вариантов быстро становится трудоёмким, а последствия неудачного выбора проявляются уже после реализации и развёртывания [1, 6].

Использование искусственного интеллекта (ИИ) позволяет *рассматривать архитектурное проектирование как поиск и оценку вариантов*. Языковая модель способна извлечь предполагаемые сервисы и связи из описаний требований, предложить несколько архитектурных кандидатов и сформировать объяснение проектных решений. При этом она не обязана обладать точной внутренней моделью очередей и реального поведения системы. Поэтому перспективна не замена архитекторов языковой моделью, а *гибридный процесс*: модель порождает возможные решения, формальные ограничения отбрасывают недопустимые варианты, а аналитические и имитационные методы оценивают количественные последствия. Архитектор сохраняет ответственность за исходные допущения и окончательный выбор.

Это направление поддерживается современными исследованиями. Обзор Esposito и соавторов отмечает расширение применения генеративного ИИ к этапам перехода от требований к архитектуре и от архитектуры к коду, одновременно указывая на недостаток строгой проверки генерируемых решений [7]. Систематический обзор Schmid и соавторов выделяет проверку соответствия архитектуры заданным свойствам среди недостаточно изученных задач [8]. Метод ArchiLLM показывает возможность получать предварительные микросервисные архитектуры из пользовательских сценариев до генерации кода [9]. Исследование проектирования с опорой на атрибуты качества также подчёркивает необходимость участия специалиста в сложных архитектурных решениях [10].

Таким образом, автоматизированный анализ вариантов может стать важной частью последующего развития программной инженерии, особенно при распространении средств генерации кода. Но тезис об исторической неизбежности полной автоматизации был бы необоснованным. Проверяемый научный вопрос заключается в том, *когда* и *насколько* сочетание LLM с независимой проверкой снижает ошибку проектирования по сравнению с альтернативными подходами.

Исследовательский вопрос: при каких нагрузках и архитектурных особенностях аналитическая и имитационная проверка снижает частоту ошибочного принятия архитектуры, сгенерированной LLM, и насколько диагностическая обратная связь помогает различным, включая локальные, языковым моделям исправлять выявленные нарушения?

2 Цель и задачи исследования

Цель — разработать и экспериментально оценить метод проверки микросервисных архитектур, создаваемых большими языковыми моделями, по структурным и количественным нефункциональным ограничениям.

Поставлены следующие задачи:

1. Формализовать граф архитектуры, профиль нагрузки, проверяемые ограничения и границы применимости оценочных моделей.

2. Разделить структурные, аналитические и имитационные проверки, включая явное указание случаев, когда заключение получить нельзя.

3. Сформировать набор контрольных сценариев с различными нагрузками, очередями и маршрутизацией, включая заведомо ошибочные архитектуры.

4. Сравнить одношаговую генерацию и итеративное исправление с разными видами обратной связи на одинаковых сценариях и моделях LLM.

5. Измерить риск ложного подтверждения соответствия, точность численных оценок, долю неопределённых случаев и вычислительные затраты.

Для последующего экспериментального исследования сформулированы следующие гипотезы:

1. Диагностическая обратная связь при заданном пределе числа итераций повышает долю структурно и функционально допустимых архитектур по сравнению с однократной генерацией.

2. Аналитические модели очередей позволяют обнаруживать перегрузку с небольшими вычислительными затратами при соблюдении их предпосылок, но не заменяют проверку перцентильных ограничений для произвольного профиля нагрузки.

3. Дискретно-событийное моделирование позволяет выявить дополнительные нарушения количественных требований в сценариях с конечными очередями, нестационарными потоками и сложной маршрутизацией, но увеличивает время оценки и само зависит от корректности предположений.

Эти утверждения являются *проверяемыми предположениями*, а не полученными результатами. Их необходимо сопоставить с независимыми эталонами и статистической неопределённостью.

3 Анализ аналогов и предшествующих исследований

Для сопоставления с разрабатываемым подходом рассмотрены четыре группы работ: генерация моделей и архитектурных представлений средствами LLM; мультиагентное проектирование; моделирование производительности микросервисов; аналитические модели очередей. Сравнение проводится по конкретным функциям: что является входом и выходом метода, как проверяется результат, учитываются ли характеристики нагрузки и может ли результат проверки изменить сгенерированную архитектуру. Такое разделение важно, поскольку генерация диаграммы, имитация уже заданной системы и тестирование развернутого приложения решают разные задачи.

3.1 Генерация моделей и архитектурных артефактов средствами LLM

Подход «диаграмма как код» (Diagram-as-Code) позволяет хранить архитектурные представления в текстовом виде и воспроизводить диаграммы из исходного описания; книга С. Брауна систематизирует практику визуализации и документирования архитектуры, в том числе с использованием C4 [11]. Однако сам факт представления модели в виде PlantUML, Mermaid или предметно-ориентированного языка (Domain-Specific Language, DSL) не гарантирует семантической корректности: нотация решает задачу представления модели,

но правила допустимости связей и выполнение ограничений производительности должны проверяться отдельно. Обзор Hou et al. систематизирует применение LLM в программной инженерии, но не является отдельным методом синтеза и оптимизации микросервисной топологии [3].

В работе NOMAD предложен мультиагентный конвейер генерации диаграмм классов на унифицированном языке моделирования (Unified Modeling Language, UML) из текстовых требований: отдельные агенты извлекают сущности, определяют отношения, интегрируют модель в промежуточное представление в формате JSON и формируют PlantUML; также предусмотрен отдельный агент-валидатор. Авторы оценивают систему на примере Northwind и на наборе заданий, созданных людьми, а ошибки классифицируют как структурные, связанные с отношениями и семантические [12]. Поэтому нельзя утверждать, что предшествующие работы вообще не используют промежуточные представления или проверку. Отличие рассматриваемой задачи заключается в другом объекте проверки: в NOMAD основной артефакт — UML-модель предметной области, тогда как здесь рассматривается топология микросервисов с типами инфраструктурных узлов, допустимыми направленными связями, параметрами пропускной способности, очередей и стоимости. В опубликованном описании NOMAD проверка не представлена как цикл, оптимизирующий топологию по результатам расчета NFR или имитации пикового трафика.

Ferrari et al. исследуют генерацию UML-диаграмм последовательностей из 28 документов с требованиями и отмечают, что полнота и корректность результата ухудшаются, в частности, при неоднозначных или противоречивых требованиях [13]. Эта работа важна как эмпирическое свидетельство того, что синтаксически приемлемый диаграммный артефакт не обязательно точно отражает требования. При этом объект исследования отличается от предлагаемого: авторы анализируют диаграммы последовательностей, а не нагрузочную состоятельность топологии микросервисов.

ArchiGen рассматривает обратное направление преобразования: из UML-диаграмм формируются проектные документы, описывающие архитектурное решение (Software Solution Blueprint). Это полезный аналог автоматизации перехода между архитектурными артефактами, но он не совпадает с генерацией топологии из исходных требований и проверкой ее поведения под нагрузкой [14].

Более близки к архитектурному синтезу системы MAAD и C4Modeler. MAAD распределяет проектирование между агентами анализа требований (Analyst), моделирования (Modeler), проектирования (Designer) и оценки (Evaluator) и формирует архитектурные чертежи и отчеты об оценке атрибутов качества; в опубликованной работе также приведено сравнение с MetaGPT [15]. C4Modeler использует диалог ролевых агентов для построения представлений C4 Context, Container и Component, сочетая детерминированные проверки структуры и правил C4 с семантической оценкой другой языковой моделью (LLM-as-a-Judge; LLM в роли эксперта) [16]. Следовательно, наличие мультиагентной генерации архитектуры и структурной проверки само по себе не является отличительным признаком настоящего исследования. Существенный предмет сравнения — включается ли в тот же контур формализованная проверка количественных ограничений по заданному профилю нагрузки и используются ли диагностические результаты для целенаправленного исправления графа. В описаниях MAAD и C4Modeler такой связанный цикл с моделированием очередей или дискретно-событийной симуляцией микросервисов не указан.

3.2 Моделирование производительности и пиковых нагрузок микросервисов

PerfSim — дискретно-событийный симулятор цепочек облачных микросервисов. Он позволяет задавать сценарии, вычислять показатели, включая среднее время ответа, и исследо-

вать различные конфигурации ресурсов и взаимодействий; авторы сообщают о точности 81–99% для оценки среднего времени ответа в рассмотренных сценариях [17]. Это уже решает важную часть задачи оценки производительности до проведения полноценного развертывания. При этом PerfSim не является генератором архитектур: его сценарий и модели характеристик микросервисов задаются извне. Следовательно, он может выступать симуляционным оценщиком в контуре проектирования, но не заменяет этап синтеза и исправления топологии.

CloudNativeSim также предназначен для моделирования приложений, изначально ориентированных на облачную инфраструктуру (cloud-native) с микросервисной архитектурой. Он поддерживает динамические модели, пользовательские сценарии, интерфейсы политик планирования и оценку качества обслуживания (Quality of Service, QoS); в опубликованном кейсе авторы сообщают о точности выше 94,5% по времени отклика и демонстрируют анализ политик масштабирования [18]. Эти результаты относятся к описанной авторами экспериментальной конфигурации и не должны интерпретироваться как универсальная гарантия точности для произвольной системы. По сравнению с исследуемым подходом CloudNativeSim сосредоточен на моделировании и оценке уже заданного приложения и политик ресурсов, а не на генерации топологии LLM с последующей коррекцией по диагностике модели.

Более ранний и методологически важный пример — Palladio Component Model. Модель задает компоненты и их параметры, контекст использования и характеристики ресурсов, после чего инструмент моделирования рассчитывает показатели производительности для архитектурных вариантов [6]. Работа показывает, что параметрические профили компонентов и симуляция архитектуры задолго до появления LLM уже использовались для поддержки архитектурных решений. Поэтому новизну нельзя формулировать как первое применение моделей производительности к программной архитектуре. В работе Khazaei et al. построена многоуровневая модель производительности микросервисной платформы, применяемая для анализа provisioning и capacity planning, включая сценарии «что будет, если» [19]. Эти исследования дают основу для моделирования ресурсов и системных эффектов, но не описывают LLM-контур генерации и целевого исправления архитектурного графа.

DeathStarBench решает другую, дополняющую задачу: это открытый набор реальных микросервисных приложений и нагрузочных сценариев, предназначенный для исследования сквозного поведения, использования ресурсов и последствий архитектурных решений [20]. В отличие от PerfSim и CloudNativeSim, это прежде всего контрольный набор для испытаний (benchmark) для экспериментов с развернутыми системами, а не самостоятельный симулятор. Он полезен как эмпирическая база для проверки того, насколько модель производительности соответствует измерениям реального приложения. Сам по себе DeathStarBench не генерирует архитектуру по требованиям и не выполняет автоматический ремонт топологии.

Таким образом, симуляция микросервисов, профили ресурсов и оценка поведения при нагрузке уже являются исследованными направлениями. Предмет настоящего исследования находится не в создании нового симулятора, а в организации единого контура, в котором генерация архитектурного графа, формализованная проверка NFR и расчетная оценка поведения при нагрузке связаны с диагностикой нарушений и последующим исправлением архитектурного решения. Отличительным свойством в таком случае является не наличие симулятора само по себе, а способ использования его результатов в процедуре выбора и коррекции топологии.

3.3 Мультиагентные системы в программной инженерии и архитектурном проектировании

ChatDev организует разработку ПО как взаимодействие специализированных агентов, выполняющих задачи проектирования, программирования и тестирования, и формализует обмен между ними через последовательности диалогов [21]. MetaGPT переносит в мультиагентный процесс стандартные процедуры разработки и распределяет роли между агентами, стремясь снизить ошибки при последовательной передаче результатов [22]. Эти работы обосновывают использование специализации ролей и промежуточных артефактов, однако их центральная задача — процесс разработки ПО в целом, а не количественная оптимизация топологии микросервисов при ограничениях задержки, пропускной способности и стоимости.

Обзор Liu et al. классифицирует исследования LLM-агентов для программной инженерии по задачам разработки и свойствам агентов, а также описывает открытые проблемы направления [4]. На основании обзора некорректно утверждать, что агентные системы в целом не обладают внешней верификацией: отдельные работы, включая NOMAD и C4Modeler, уже содержат явные механизмы проверки. Более точное различие состоит в характере критерия и его включении в цикл работы. Проверка кода, синтаксиса UML или соблюдения правил C4 не эквивалентна детерминированной проверке инфраструктурного бюджета и ограничений времени ответа, очередей или хвостовой задержки по заданному профилю трафика.

Для исследуемого подхода принципиальна связка из трех частей: (1) LLM формирует или корректирует только архитектурное представление в заданной схеме; (2) обычные программные валидаторы проверяют структурные инварианты и предметные ограничения, не полагаясь на самооценку модели; (3) диагностические нарушения возвращаются на ограниченный шаг исправления. В таком описании мультиагентность не является обязательным условием метода: методологически важнее разделить вероятностную генерацию и детерминированное принятие решения о допустимости результата.

3.4 Теория массового обслуживания и модели производительности

Классическая теория массового обслуживания дает аналитические оценки загрузки, ожидания в очереди и времени пребывания заявки в системе [2, 23]. Сети Джексона формализуют потоки заявок между несколькими узлами и допускают разложение стационарного распределения на узловые распределения при выполнении соответствующих условий: открытая сеть, пуассоновские внешние поступления, марковская маршрутизация и экспоненциальные времена обслуживания в стандартной модели [24]. Это позволяет анализировать сеть не как набор независимых диаграмм, а как систему взаимосвязанных очередей.

Для узла с m_i параллельными обслуживающими устройствами необходимым условием стационарности стандартной модели является

$$\rho_i = \frac{\lambda_i}{m_i \mu_i} < 1,$$

где λ_i — эффективная интенсивность поступления заявок, а μ_i — интенсивность обслуживания одного устройства. Для очередей с произвольными распределениями времени поступления и обслуживания применимы иные модели и приближения, в частности аппроксимация Кингмана для G/G/1 [25]. Нельзя без дополнительных условий одновременно приписывать произвольной сети M/G/m свойство произведения распределений, характерное для классической сети Джексона.

В контексте микросервисов аналитическая модель полезна для быстрой предварительной оценки и выявления узлов, приближающихся к насыщению. Однако ее результат зависит от предположений о распределениях, маршрутизации, количестве параллельных обработчиков и стационарности потока. Резкие всплески, fan-out, повторные попытки, ограниченные буферы, совместное использование CPU или пула соединений, autoscaling и корреляции между вызовами могут нарушать эти упрощения. Поэтому теория очередей не заменяет моделирование переходных процессов или измерения: аналитические оценки целесообразно использовать как отдельный уровень проверки, а пиковые и нестационарные сценарии исследовать симуляцией и, когда это возможно, сверять с нагрузочным стендом.

3.5 Сопоставление с исследуемым подходом

Таблица 1 суммирует не общие достоинства публикаций, а функции, непосредственно относящиеся к предмету исследования. Утверждения о возможностях аналогов ограничены функциями, описанными в соответствующих публикациях.

По рассмотренным источникам не следует заявлять, что ранее не существовало генерации диаграмм LLM, мультиагентного проектирования, проверки архитектурных моделей, имитации микросервисных систем или применения теории очередей: все эти направления имеют опубликованные аналоги. Более узкое и проверяемое позиционирование состоит в исследовании интегрированного контура: генерация типизированного графа микросервисной архитектуры, формальные правила допустимых связей, проверка NFR и диагностически направленный ремонт. Если симуляция пиковых нагрузок включена в этот контур, ее потенциальное отличие заключается в передаче выявленных узких мест обратно на этап коррекции топологии; эффективность такой связки должна оцениваться экспериментально.

4 Постановка задачи

Входной сценарий задаётся кортежем

$$\mathcal{S} = \langle \mathcal{D}, \mathcal{F}, \mathcal{W}, \mathcal{R} \rangle, \tag{1}$$

где $\mathcal{D}$ — текстовое описание предметной области, $\mathcal{F}$ — функциональные сценарии, $\mathcal{W}$ — профиль нагрузки, $\mathcal{R}$ — ограничения на количественные показатели. Интенсивность λ измеряется в запросах в секунду (requests per second, RPS).

Синтезируемая архитектура представлена ориентированным графом $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Вершина $v \in \mathcal{V}$ содержит тип компонента, число реплик r_v, число независимых каналов обслуживания c_v, предел длины очереди q_v и распределение времени обработки. Ребро $e \in \mathcal{E}$ определяет направление и тип взаимодействия, маршрутизацию и сетевую задержку. В частности, максимальная конкурентность *не равна* автоматически числу независимых каналов обслуживания.

Структурный предикат $V_{\text{str}}(\mathcal{G})$ отражает соблюдение правил связности, типов и разрешённых переходов. Для каждого количественного требования оценщик должен формировать один из четырёх выводов: *выполнено*, *нарушено*, *недостаточно оснований* или *не оценивалось*. Последний вывод нужен, если модель не содержит параметров для расчёта требуемого показателя.

Для ограничения $L_p \le L_{\max}$ на p-й перцентиль клиентской задержки имитация должна сообщать оценку $\widehat{L}_p$ и доверительный интервал $[L_p^-, L_p^+]$. Если $L_p^+ \le L_{\max}$, допускается заключение о выполнении требования *в принятой модели и на заданном уровне доверия*. Если $L_p^- > L_{\max}$, фиксируется нарушение. При пересечении интервала с порогом результат

Таблица 1: Сопоставление предшествующих подходов с исследуемым методом

Подход	**Что уже решается**	**Отличие относительно исследуемой задачи**
NOMAD [12]	Мультиагентная генерация UML-классов, промежуточная JSON-модель, отдельный агент-валидатор, классификация ошибок.	Предметная модель классов; в опубликованном описании нет проверки микросервисной топологии по профилям нагрузки и ремонтов на основе расчета производительности.
MAAD и C4Modeler [15,16]	Ролевое проектирование архитектуры; генерация C4; структурные проверки и оценка архитектурного качества.	Основное внимание уделено созданию и оценке архитектурных артефактов; связанный цикл «нарушение NFR по модели нагрузки — локализация узкого места — коррекция топологии» в описанных методах не представлен.
PerfSim и CloudNativeSim [17, 18]	Дискретно-событийная симуляция микросервисных цепочек, оценка задержек и QoS, исследование вариантов ресурсов и политик масштабирования.	Симуляторы оценивают заданные модели/сценарии; сами по себе они не синтезируют требованиями управляемый архитектурный граф и не формируют LLM-предложения для исправления ошибок этого графа.
Palladio и модель Khazaei et al. [6,19]	Параметрические модели компонентов, анализ ресурсов, симуляция и планирование ёмкости.	Подтверждают зрелость модельно-ориентированного анализа производительности, но не описывают связку LLM-генерации микросервисной топологии с направленным циклом исправления.
DeathStarBench [20]	Нагрузочные эксперименты на развернутых микросервисных приложениях и анализ сквозных эффектов.	Это эмпирический бенчмарк, а не генератор архитектур или симулятор; полезен для проверки внешней валидности предсказаний.
Исследуемый подход AutoSystemDesign	Структурированное представление архитектуры как графа; проверка ссылочной целостности, топологических правил и разрешенных типов связей; ограничение бюджета и проверка NFR; направленный режим INITIAL/REPAIR с передачей диагностик; экспорт в C4 Container.	Исследуется интеграция генерации и детерминированной верификации в замкнутый контур. Целевой уровень развития метода — использование количественной оценки производительности, включая моделирование очередей и нестационарной нагрузки, для локализации нарушений и корректировки топологии.

остаётся статистически неопределённым. Нельзя делать вывод по единственному случайному прогону или заменять перцентиль средним временем ответа.

Минимизация инфраструктурных затрат $C(\mathcal{G})$ при удовлетворении ограничений является дополнительной поисковой задачей. Итеративная генерация может получить допустимый кандидат, но без доказательства оптимальности не имеет оснований объявлять найденное решение глобальным минимумом.

5 Метод проверки и итеративной корректировки

Первоначально языковая модель преобразует сценарий $\mathcal{S}$ в кандидат $\mathcal{G}_0$. Проверка возвращает структурированные причины ошибок $\mathcal{F}_k$. По ним модель может предложить исправленный кандидат:

$$\mathcal{G}_{k+1} = \mathrm{LLM}(\mathcal{S}, \mathcal{G}_k, \mathcal{F}_k), \qquad 0 \le k < K, \tag{2}$$

где K — предел числа итераций. Эта запись описывает алгоритм, но *не доказывает его сходимость*. Цикл завершается после успешных проверок, исчерпания ограничений или обнаружения неподдерживаемых исходных условий.

5.1 Структурные проверки и диагностическая обратная связь

На первом уровне проверяются существование узлов, допустимость связей, достижимость необходимых компонентов и отсутствие запрещённых циклов. Ошибки форматирования также фиксируются, но не смешиваются с содержательными нарушениями. Формат JSON (JavaScript Object Notation; текстовое представление структурированных данных) облегчает передачу графа между этапами, но сам по себе не гарантирует его смысловую корректность.

Модель архитектурных представлений C4 (Context, Containers, Components, Code; контекст, контейнеры, компоненты, код) применяется для визуализации. Корректная C4-диаграмма не является свидетельством выполнения ограничений по задержке и пропускной способности. Адресное сообщение об ошибке, например «ребро указывает на отсутствующий узел», позволяет проверить эффект целенаправленной коррекции по сравнению с полным повторным генерированием.

5.2 Аналитические и имитационные проверки

После установления структурной допустимости рассчитываются поступающие на узлы потоки и аналитические показатели загрузки. Расчёт допустим только при известной дисциплине обслуживания, числе каналов и распределениях поступлений и обслуживания. Имитация расширяет класс исследуемых ситуаций: можно воспроизвести конечные очереди, случайные времена обработки, пиковые нагрузки и отказы в приёме запросов.

Каждый уровень возвращает не только значение показателя, но и его предпосылки. Неизвестное число каналов, неподдержанная семантика асинхронного вызова, отсутствие модели отказов или неполный клиентский путь не должны автоматически приводить к заключению «требования выполнены». Подробное сравнение этих уровней составляет предмет планируемого эксперимента.

6 Структурная допустимость архитектурного графа

Проверка корректности архитектуры задаётся совокупностью формальных ограничений, независимых от конкретной языковой модели. Для графа $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ примерами таких

ограничений служат:

1. **Уникальность идентификаторов узлов**: разные вершины имеют разные идентификаторы:
$$\forall u, v \in \mathcal{V}, \quad u \neq v \Rightarrow id(u) \neq id(v). \tag{3}$$

2. **Ссылочная целостность**: оба конца каждого ребра принадлежат графу:
$$\forall (u, v) \in \mathcal{E}, \quad u \in \mathcal{V} \wedge v \in \mathcal{V}. \tag{4}$$

3. **Достижимость**: от клиентского входа достижимы узлы, необходимые для выполнения соответствующего функционального сценария.

4. **Отсутствие циклов в выбранной модели синхронных вызовов**: для рассматриваемой аналитической модели предполагается ориентированный ациклический граф (Directed Acyclic Graph, DAG). Это *ограничение модели*, а не утверждение, будто любой цикл между микросервисами неизбежно приводит к взаимоблокировке.

6.1 Правила допустимости взаимодействий

Пусть $\mathcal{T}_{\text{allow}}$ — множество разрешённых пар типов компонентов, а $\tau(v)$ — тип вершины. Тогда
$$\forall (u, v) \in \mathcal{E} : \quad (\tau(u), \tau(v)) \in \mathcal{T}_{\text{allow}}. \tag{5}$$

Например, политика конкретного проекта может разрешать связи от балансировщика нагрузки к вычислительному сервису и от вычислительного сервиса к базе данных, запрещая клиенту прямое обращение к базе. Такое правило выражает *принятую архитектурную политику*; оно не является универсальным запретом для всех распределённых систем.

Допустимый путь в рассматриваемой политике
Клиент $\longrightarrow$ Балансировщик $\longrightarrow$ Сервис $\longrightarrow$ База данных

Пример выявляемого нарушения
Клиент $\longrightarrow$ База данных
Нарушена заданная матрица разрешённых типов взаимодействий.

Рис. 1: Пример структурной проверки по принятой политике связей

Структурная допустимость не гарантирует устойчивости по нагрузке. Даже ациклическая и связная топология может иметь узкое место в единственной базе данных или перегруженном сервисе. Именно поэтому следующий уровень исследования оценивает поведение запросов и очередей, а не только отношения между вершинами.

7 Проверяемые нефункциональные требования

Для детерминированной характеристики $m(\mathcal{G})$, например стоимости известного набора реплик, проверка порога θ имеет вид $m(\mathcal{G}) \leq \theta$. Для случайной задержки, размера очереди и потерь необходимо явно задать окно наблюдения, статистический показатель и допустимую неопределённость оценки.

Ограничение стоимости можно записать как
$$C(\mathcal{G}) = \sum_{v \in \mathcal{V}_{\text{billable}}} r_v c_v^{\text{month}}, \tag{6}$$

где $\mathcal{V}_{\text{billable}}$ — оплачиваемые компоненты, r_v — число реплик, c_v^{month} — их заданная месячная стоимость. Модельные стоимости не должны выдаваться за реальные тарифы облачных провайдеров без источника и даты.

Соглашение об уровне обслуживания (Service Level Agreement, SLA) и целевой показатель его качества (Service Level Objective, SLO) не равны доказательству эксплуатационной надёжности. Даже статистически уверенный прогноз выполнения ограничения на 99-й перцентиль задержки относится лишь к заданной модели и нагрузке. Доступность нельзя вычислить по задержкам без модели отказов и восстановления.

8 Аналитическая проверка методами теории массового обслуживания

Нотация Кендалла M/M/1 означает пуассоновский поток поступающих запросов (экспоненциальные интервалы между ними), экспоненциальное обслуживание и один обслуживающий канал. В M/M/c количество одинаковых независимых каналов равно c. Предполагаются обслуживание в порядке прихода (First In, First Out, FIFO; первым пришёл — первым обслужен), бесконечная очередь и стационарный режим [2, 23].

Пусть λ — интенсивность поступлений (запросов/с), μ — интенсивность обслуживания одного канала (запросов/с), а ρ — доля его загрузки. Для устойчивой M/M/1

$$\rho = \frac{\lambda}{\mu} < 1, \qquad W = \frac{1}{\mu - \lambda}, \qquad W_q = W - \frac{1}{\mu}, \tag{7}$$

где W — среднее время нахождения запроса в системе (ожидание и обслуживание), W_q — среднее время ожидания в очереди.

Расчётный пример. Для $\lambda = 10$ запросов/с и среднего обслуживания 50 мс имеем $\mu = 20$ запросов/с, $\rho = 0{,}5$, $W = 100$ мс и $W_q = 50$ мс. Это независимо проверяемые аналитические значения, а не результаты уже проведённого эксперимента.

Для M/M/c обозначим $a = \lambda/\mu$, $\rho = a/c < 1$. По формуле Эрланга C вероятность того, что запрос будет ожидать, равна

$$P_{\text{wait}} = \frac{\dfrac{a^c}{c!(1-\rho)}}{\displaystyle\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c!(1-\rho)}}. \tag{8}$$

Среднее ожидание и среднее полное время ответа:

$$W_q = \frac{P_{\text{wait}}}{c\mu - \lambda}, \qquad W = W_q + \frac{1}{\mu}. \tag{9}$$

Конечные очереди, произвольные времена обслуживания, нестационарные пики и синхронные разветвления требуют других моделей. Приближённые формулы для очередей G/G/c могут помочь оценить среднее ожидание, но их нельзя напрямую трактовать как 95-й или 99-й перцентиль клиентской задержки.

9 Дискретно-событийная проверка архитектуры

В имитационной модели последовательно воспроизводятся события поступления запроса, постановки в очередь, начала и окончания обработки и переходов между сервисами. Виртуальное время моделирования не совпадает с длительностью выполнения программы.

Для реализации эксперимента предполагается использовать SimPy: библиотеку событий, процессов и ограниченных ресурсов [5].

Модель каждого компонента должна учитывать число реплик и *отдельное* число обслуживающих каналов, дисциплину и длину очереди. Максимальную конкурентность нельзя без обоснования приравнивать к количеству каналов. Первый планируемый вариант использует пуассоновский входной поток и случайное время обслуживания. При среднем времени $m > 0$ и стандартном отклонении s гамма-распределение может задаваться параметрами

$$\alpha = \left(\frac{m}{s}\right)^2, \qquad \theta = \frac{s^2}{m}, \qquad s > 0. \tag{10}$$

Здесь α — параметр формы, θ — параметр масштаба; при $s = 0$ время обслуживания постоянно. Это *предположенная* форма распределения, а не следствие двух известных моментов.

Необходимо раздельно учитывать предложенную нагрузку, завершённые, отклонённые и незавершённые к границе окна заявки. Перцентиль времени ответа по *только завершённым* операциям не гарантирует выполнение SLO, если часть запросов была потеряна. Выводы «выполнено» и «нарушено» требуют независимых повторений моделирования с разными случайными начальными состояниями, прогревом и зафиксированным окном измерения. Статистическая неопределённость оценивается по независимым повторениям, а не путём ошибочного принятия всех событий одного прогона за независимые эксперименты.

10 Методика планируемого экспериментального исследования

10.1 Проверка оценщиков отдельно от языковой модели

Эксперимент A проводится на фиксированных архитектурах без вызова LLM. Для M/M/1 и M/M/c сопоставляются точные аналитические значения с результатами достаточно длительной имитации. Для более сложных систем заранее подготавливается независимый эталон: отдельная реализация симуляции с установленной семантикой либо нагрузочное тестирование развёрнутого прототипа. Проверять точность симулятора сравнением с самим собой нельзя.

Сценарии будут варьировать интенсивность и характер нагрузки, число обслуживающих каналов, ёмкость очереди, вариативность обслуживания, число реплик и способ маршрутизации. Обязательно включаются перегрузки, пиковые режимы, неизвестные параметры и неподдерживаемые типы взаимодействий. Помимо погрешностей численных оценок фиксируется доля ситуаций, в которых метод честно отказывается делать вывод.

10.2 Влияние проверок на генерацию LLM

Эксперимент B использует одинаковые текстовые требования и профили нагрузки для пяти сравниваемых способов проектирования:

1. Однократная генерация без обратной связи.
2. Однократная генерация с последующей проверкой, но без исправления.
3. Итеративная генерация с диагностикой нарушений структуры.
4. Итеративная генерация со структурной и аналитической проверкой ТМО.

5. Итеративная генерация со структурной, аналитической и имитационной проверками.

Внешняя проверка итоговой архитектуры должна быть одинаковой для всех вариантов, а её результат нельзя подменять самооценкой модели. Сравниваются как локально запускаемые LLM, так и другие доступные модели. Список точных версий, аппаратная конфигурация, температура, формат ответа, лимиты токенов и число итераций будут зафиксированы *до* основного исследования.

10.3 Показатели и статистическая процедура

Главным показателем безопасности проверки выбрана частота *ложного подтверждения выполнения требований*. Пусть N_{viol} — количество сценариев с независимо установленными нарушениями, а $N_{\text{false pass}}$ — число сценариев, ошибочно признанных удовлетворяющими ограничениям. Тогда

$$R_{\text{false pass}} = \frac{N_{\text{false pass}}}{N_{\text{viol}}}, \qquad N_{\text{viol}} > 0. \tag{11}$$

Дополнительно измеряются доля структурно допустимых графов, ошибки среднего и перцентильного времени ответа, частота отказов, пропускная способность, доля оценённых сценариев, число итераций, расход токенов и время расчёта. Если метод возвращает «не оценивалось», такой случай нельзя исключать из отчёта о покрытии.

Для языковых моделей предусматриваются независимые повторы генерации; для симулятора — независимые репликации с контролируемыми случайными начальными состояниями. Различия между сравниваемыми вариантами будут оцениваться с учётом парности сценариев и доверительных интервалов. Объём выборки, первичные критерии и правило обработки множественных сравнений подлежат фиксации до изучения итогов.

11 Состояние работы и ограничения

В проекте AutoSystemDesign уже имеются средства представления и структурной проверки архитектурных графов. Аналитическая проверка на основе TMO и моделирование SimPy разрабатываются в отдельных pull request и не считаются полностью интегрированными и экспериментально принятыми. В статье намеренно разделены формальная постановка, существующие компоненты стенда и *запланированные* численные опыты.

Основные угрозы достоверности: различие между принятым семейством распределений и реальным трафиком, неопределённость маршрутизации, неполная модель отказов, погрешность эталона и возможное попадание тестовых требований в обучающие данные LLM. До проведения запланированных опытов выводы о статистически значимом превосходстве какого-либо метода или модели делать нельзя.

12 Заключение

Сформулирована задача сравнительной проверки микросервисных архитектур, создаваемых большими языковыми моделями, с разграничением структурных ограничений, аналитических оценок очередей и дискретно-событийного моделирования. Показано на уровне математической постановки, что среднее время ожидания в стационарной модели M/M/1 или M/M/c не является универсальным свидетельством выполнения перцентильных требований при конечных очередях, перегрузках и сложной маршрутизации.

Предложена программа двух взаимосвязанных исследований: независимая проверка численных оценщиков на контрольных архитектурах и сравнение способов генерации с

различной диагностической обратной связью. Основными ожидаемыми результатами будущих экспериментов являются количественные оценки риска ложного подтверждения соответствия, покрытия проверок, точности задержек, вычислительных расходов и чувствительности разных LLM к обратной связи.

На текущем этапе экспериментальных результатов, подтверждающих выдвинутые гипотезы, нет. Исследовательская ценность последующей работы будет определяться воспроизводимостью сценариев, независимостью эталонов и статистически обоснованным сравнением методов, а не самим фактом реализации программного комплекса.

Список литературы

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley Professional, 3rd ed., 2015.

[2] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[3] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.

[4] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.

[5] "Simpy: Discrete event simulation for python." https://simpy.readthedocs.io/en/stable/. Official documentation, accessed October 22, 2026.

[6] S. Becker, H. Koziolek, and R. H. Reussner, "The palladio component model for model-driven performance prediction," *Journal of Systems and Software*, vol. 82, no. 1, pp. 3–22, 2009.

[7] M. Esposito, X. Li, S. Moreschini, N. Ahmad, T. Cerny, K. Vaidhyanathan, V. Lenarduzzi, and D. Taibi, "Generative ai for software architecture: Applications, challenges, and future directions," *Journal of Systems and Software*, vol. 231, p. 112607, 2026.

[8] L. Schmid, T. Hey, M. Armbruster, S. Corallo, D. Fuchss, J. Keim, H. Liu, and A. Koziolek, "Software architecture meets llms: A systematic literature review," 2025.

[9] M. Calamo, F. Monti, F. Spaziani, F. Leotta, and M. Mecella, "From user stories to architectures: Using llm-powered agents to design and improve microservice-based software," *IEEE Software*, 2026.

[10] H. Cervantes, R. Kazman, and Y. Cai, "Better together: An llm-assisted approach to designing software architectures using attribute-driven design," *IEEE Transactions on Software Engineering*, 2026.

[11] S. Brown, *Software Architecture for Developers: Visualise, Document and Explore Your Software Architecture*. Leanpub, 2018.

[12] P. Giannouris and S. Ananiadou, "Nomad: A multi-agent llm system for uml class diagram generation from natural language requirements," in *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering (MODELSWARD)*, pp. 257–264, SciTePress, 2026.

[13] A. Ferrari, S. Abualhaija, and C. Arora, "Model generation with llms: From requirements to uml sequence diagrams," in *Proceedings of the 32nd IEEE International Requirements Engineering Conference Workshops (REW 2024)*, pp. 291–300, 2024.

[14] M. Megahed, "Archigen: An llm-augmented framework for automated generation of software solution blueprint documents from uml diagrams," in *2026 Intelligent Methods, Systems, and Applications (IMSA)*, IEEE, 2026.

[15] R. Li, Y. Zhang, X. Zhou, P. Liang, W. Sun, J. Xuan, Z. Jin, and Y. Liu, "Maad: Automate software architecture design through knowledge-driven multi-agent collaboration," *arXiv preprint arXiv:2507.21382*, 2025.

[16] K. Szczepanik and J. A. Chudziak, "Collaborative llm agents for c4 software architecture design automation," *arXiv preprint arXiv:2510.22787*, 2025.

[17] M. Gokan Khan, J. Taheri, A. Al-Dulaimy, and A. Kassler, "Perfsim: A performance simulator for cloud native microservice chains," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1395–1413, 2023.

[18] J. Wu, M. Xu, Y. He, K. Ye, and C. Xu, "Cloudnativesim: A toolkit for modeling and simulation of cloud-native applications," *arXiv preprint arXiv:2409.05093*, 2024.

[19] H. Khazaei, N. Mahmoudi, C. Barna, and M. Litoiu, "Performance modeling of microservice platforms," *IEEE Transactions on Cloud Computing*, vol. 10, no. 4, pp. 2848–2862, 2022.

[20] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, C. Hu, B. Ritchken, B. Jackson, *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 3–18, 2019.

[21] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," *arXiv preprint arXiv:2307.07924*, 2023.

[22] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[23] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, 1975.

[24] J. R. Jackson, "Networks of waiting lines," *Operations Research*, vol. 5, no. 4, pp. 518–521, 1957.

[25] J. F. C. Kingman, "The single server queue in heavy traffic," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 57, no. 4, pp. 902–904, 1961.